

Bayesian robust regression

José G. Dias

Table of contents

1. Summary

This text discusses robust regression models that are more flexible than the one assuming Gaussian errors. It shows the flexibility of the Bayesian paradigm in this context.

2. Multiple linear regression

2.1 Cauchy distribution

The density of the Cauchy distribution $f(x; x_0, \gamma)$ is:

$$f(x; x_0, \gamma) = \frac{1}{\pi\gamma \left[1 + \left(\frac{x - x_0}{\gamma} \right)^2 \right]}$$

where x_0 and γ are the median and mean absolute deviation (MAD), respectively. The Half-Cauchy distribution is often used as prior to the standard deviation (σ).

Depict the Cauchy and Half-Cauchy distributions with median 0 and distinct gammas

```
x <- seq(-5,5,.05)

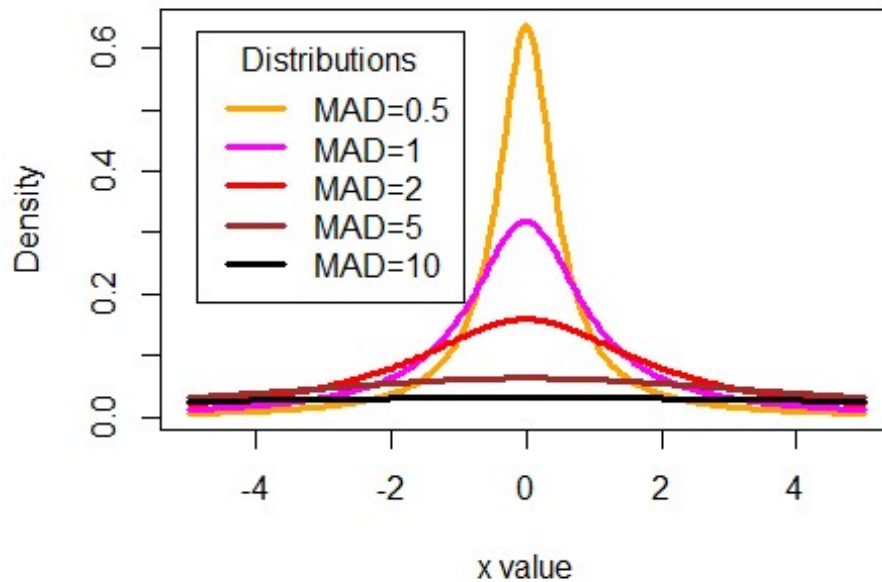
MAD <- c(0.5, 1, 2, 5, 10)
colors <- c("orange", "magenta", "red", "brown", "black")
labels <- c("MAD=0.5", "MAD=1", "MAD=2", "MAD=5", "MAD=10")
hx <- dcauchy(x,0,0.5)

plot(x, hx, type="l", lty=1, lwd = 3, xlab="x value", col=colors[1],
      ylab = "Density", main = "The Cauchy distribution")

for (i in 2:5){
  lines(x, dcauchy(x,0,MAD[i]), lwd=3, col=colors[i])
}

legend("topleft", inset=.05, title="Distributions",
      labels, lwd=3, col=colors)
```

The Cauchy distribution



And the Half-Cauchy distribution corresponds to the positive side:

Depict the Cauchy and Half-Cauchy distributions with median 0 and distinct gammas

```
x <- seq(0,5,.05)
```

```
MAD <- c(0.5, 1, 2, 5, 10)
```

```
colors <- c("orange", "magenta", "red", "brown", "black")
```

```
labels <- c("MAD=0.5", "MAD=1", "MAD=2", "MAD=5", "MAD=10")
```

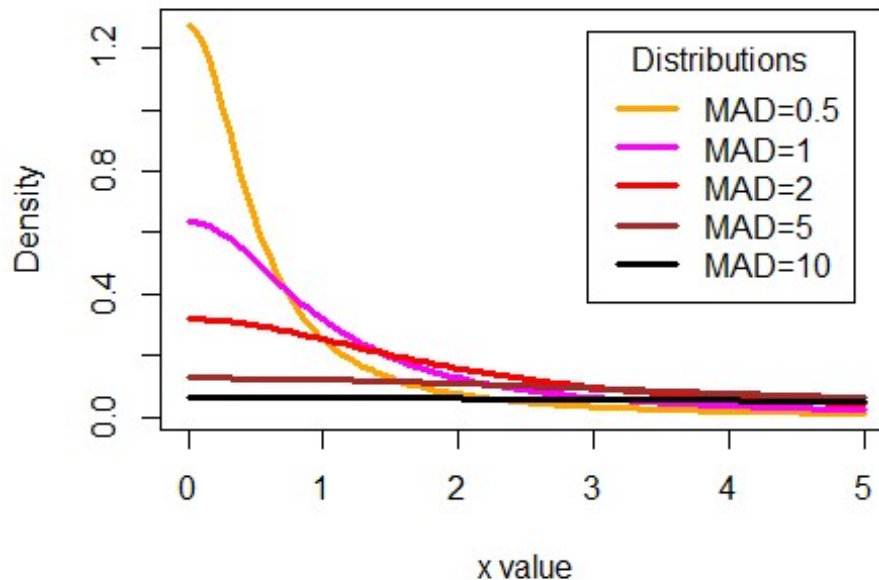
```
hx <- 2*dcauchy(x,0,0.5)
```

```
plot(x, hx, type="l", lty=1, lwd = 3, xlab="x value", col=colors[1],  
      ylab = "Density", main = "The Half-Cauchy distribution")
```

```
for (i in 2:5){  
  lines(x, 2*dcauchy(x,0,MAD[i]), lwd=3, col=colors[i])  
}
```

```
legend("topright", inset=.05, title="Distributions",  
      labels, lwd=3, col=colors)
```

The Half-Cauchy distribution



2.2 Model specification

The first model to be specified is the (classic) multiple linear regression model with normal errors (likelihood). But the prior for the dispersion is now distinct. The specification is:

$$mpg_i | cyl_i, wt_i, \beta_0, \beta_1, \beta_2, \sigma^2 \sim N(\beta_0 + \beta_1 cyl + \beta_2 wt, \sigma^2)$$
$$(\beta_0, \beta_1, \beta_2) \sim N(0, 100^2 I_3)$$

$$\sigma \sim Half\text{-}Cauchy(0,5)$$

2.3 Model estimation (MCMC)

```
#install.packages('rstan')  
library(rstan)
```

Loading required package: StanHeaders

rstan version 2.32.6 (Stan version 2.32.2)

For execution on a local, multicore CPU with excess RAM we recommend calling `options(mc.cores = parallel::detectCores())`.

To avoid recompilation of unchanged Stan programs, we recommend calling `rstan_options(auto_write = TRUE)`

For within-chain threading using ``reduce_sum()`` or ``map_rect()`` Stan functions,

```
change `threads_per_chain` option:  
rstan_options(threads_per_chain = 1)
```

Do not specify '-march=native' in 'LOCAL_CPPFLAGS' or a Makevars file

```
library(posterior)
```

This is posterior version 1.5.0

Attaching package: 'posterior'

The following objects are masked from 'package:rstan':

```
ess_bulk, ess_tail
```

The following objects are masked from 'package:stats':

```
mad, sd, var
```

The following objects are masked from 'package:base':

```
%in%, match
```

```
library(bayesplot)
```

This is bayesplot version 1.11.1

- Online documentation and vignettes at mc-stan.org/bayesplot

- bayesplot theme set to bayesplot::theme_default()

- * Does `_not_` affect other ggplot2 plots

- * See `?bayesplot_theme_set` for details on theme setting

Attaching package: 'bayesplot'

The following object is masked from 'package:posterior':

```
rhat
```

```
library(coda)
```

Attaching package: 'coda'

The following object is masked from 'package:rstan':

```
traceplot
```

```
color_scheme_set("brightblue")
```

Define the Half-Cauchy regression model in stan:

```
# Stan model code
```

```
stan_model.hc <- '
data {
  int<lower=1> n;    // Number of observations
  real mpg[n];      // Data DV: mpg
  real cyl[n];      // Data IV: cyl
  real wt[n];       // Data IV: wt
}

parameters {
  real beta0;
  real beta1;
  real beta2;
  real<lower=0> sigma;
}

model {
  // Likelihood
  for (i in 1:n)
    mpg[i] ~ normal(beta0+beta1*cyl[i] + beta2*wt[i],sigma);

  // Priors for betas
  beta0 ~ normal(0,100);
  beta1 ~ normal(0,100);
  beta2 ~ normal(0,100);

  // Prior for sigma
  sigma ~ cauchy(0,5);
}'
```

```
# Running stan code
```

```
model.hc <- stan_model(model_code = stan_model.hc)
```

```
# data set
```

```
data <- list(n = length(mtcars$mpg), mpg=mtcars$mpg, cyl = mtcars$cyl, wt =
mtcars$wt)
```

```
# Run the Stan model
```

```
stan_fit.hc <- sampling(model.hc, data = data, chains = 4,
                        iter = 4000, warmup = 1000)
```

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 1).

Chain 1:

Chain 1: Gradient evaluation took 4.7e-05 seconds

Chain 1: 1000 transitions using 10 leapfrog steps per transition would take

0.47 seconds.

Chain 1: Adjust your expectations accordingly!

Chain 1:

Chain 1:

Chain 1: Iteration: 1 / 4000 [0%] (Warmup)
Chain 1: Iteration: 400 / 4000 [10%] (Warmup)
Chain 1: Iteration: 800 / 4000 [20%] (Warmup)
Chain 1: Iteration: 1001 / 4000 [25%] (Sampling)
Chain 1: Iteration: 1400 / 4000 [35%] (Sampling)
Chain 1: Iteration: 1800 / 4000 [45%] (Sampling)
Chain 1: Iteration: 2200 / 4000 [55%] (Sampling)
Chain 1: Iteration: 2600 / 4000 [65%] (Sampling)
Chain 1: Iteration: 3000 / 4000 [75%] (Sampling)
Chain 1: Iteration: 3400 / 4000 [85%] (Sampling)
Chain 1: Iteration: 3800 / 4000 [95%] (Sampling)
Chain 1: Iteration: 4000 / 4000 [100%] (Sampling)

Chain 1:

Chain 1: Elapsed Time: 0.118 seconds (Warm-up)

Chain 1: 0.419 seconds (Sampling)

Chain 1: 0.537 seconds (Total)

Chain 1:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 2).

Chain 2:

Chain 2: Gradient evaluation took 5e-06 seconds

Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 0.05 seconds.

Chain 2: Adjust your expectations accordingly!

Chain 2:

Chain 2:

Chain 2: Iteration: 1 / 4000 [0%] (Warmup)
Chain 2: Iteration: 400 / 4000 [10%] (Warmup)
Chain 2: Iteration: 800 / 4000 [20%] (Warmup)
Chain 2: Iteration: 1001 / 4000 [25%] (Sampling)
Chain 2: Iteration: 1400 / 4000 [35%] (Sampling)
Chain 2: Iteration: 1800 / 4000 [45%] (Sampling)
Chain 2: Iteration: 2200 / 4000 [55%] (Sampling)
Chain 2: Iteration: 2600 / 4000 [65%] (Sampling)
Chain 2: Iteration: 3000 / 4000 [75%] (Sampling)
Chain 2: Iteration: 3400 / 4000 [85%] (Sampling)
Chain 2: Iteration: 3800 / 4000 [95%] (Sampling)
Chain 2: Iteration: 4000 / 4000 [100%] (Sampling)

Chain 2:

Chain 2: Elapsed Time: 0.1 seconds (Warm-up)

Chain 2: 0.294 seconds (Sampling)

Chain 2: 0.394 seconds (Total)

Chain 2:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 3).

Chain 3:

Chain 3: Gradient evaluation took 5e-06 seconds
Chain 3: 1000 transitions using 10 leapfrog steps per transition would take 0.05 seconds.

Chain 3: Adjust your expectations accordingly!

Chain 3:

Chain 3:

Chain 3: Iteration: 1 / 4000 [0%] (Warmup)
Chain 3: Iteration: 400 / 4000 [10%] (Warmup)
Chain 3: Iteration: 800 / 4000 [20%] (Warmup)
Chain 3: Iteration: 1001 / 4000 [25%] (Sampling)
Chain 3: Iteration: 1400 / 4000 [35%] (Sampling)
Chain 3: Iteration: 1800 / 4000 [45%] (Sampling)
Chain 3: Iteration: 2200 / 4000 [55%] (Sampling)
Chain 3: Iteration: 2600 / 4000 [65%] (Sampling)
Chain 3: Iteration: 3000 / 4000 [75%] (Sampling)
Chain 3: Iteration: 3400 / 4000 [85%] (Sampling)
Chain 3: Iteration: 3800 / 4000 [95%] (Sampling)
Chain 3: Iteration: 4000 / 4000 [100%] (Sampling)

Chain 3:

Chain 3: Elapsed Time: 0.1 seconds (Warm-up)

Chain 3: 0.326 seconds (Sampling)

Chain 3: 0.426 seconds (Total)

Chain 3:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 4).

Chain 4:

Chain 4: Gradient evaluation took 5e-06 seconds

Chain 4: 1000 transitions using 10 leapfrog steps per transition would take 0.05 seconds.

Chain 4: Adjust your expectations accordingly!

Chain 4:

Chain 4:

Chain 4: Iteration: 1 / 4000 [0%] (Warmup)
Chain 4: Iteration: 400 / 4000 [10%] (Warmup)
Chain 4: Iteration: 800 / 4000 [20%] (Warmup)
Chain 4: Iteration: 1001 / 4000 [25%] (Sampling)
Chain 4: Iteration: 1400 / 4000 [35%] (Sampling)
Chain 4: Iteration: 1800 / 4000 [45%] (Sampling)
Chain 4: Iteration: 2200 / 4000 [55%] (Sampling)
Chain 4: Iteration: 2600 / 4000 [65%] (Sampling)
Chain 4: Iteration: 3000 / 4000 [75%] (Sampling)
Chain 4: Iteration: 3400 / 4000 [85%] (Sampling)
Chain 4: Iteration: 3800 / 4000 [95%] (Sampling)
Chain 4: Iteration: 4000 / 4000 [100%] (Sampling)

Chain 4:

Chain 4: Elapsed Time: 0.096 seconds (Warm-up)

Chain 4: 0.275 seconds (Sampling)

Chain 4: 0.371 seconds (Total)

Chain 4:

```
# Print and plot a summary of the Stan fit
```

```
print(stan_fit.hc)
```

Inference for Stan model: anon_model.

4 chains, each with iter=4000; warmup=1000; thin=1;

post-warmup draws per chain=3000, total post-warmup draws=12000.

	mean	se_mean	sd	2.5%	25%	50%	75%	97.5%	n_eff	Rhat
beta0	39.64	0.02	1.80	36.12	38.45	39.64	40.81	43.17	6668	1
beta1	-1.50	0.01	0.44	-2.38	-1.79	-1.49	-1.21	-0.62	4663	1
beta2	-3.19	0.01	0.80	-4.80	-3.70	-3.19	-2.67	-1.62	4439	1
sigma	2.67	0.00	0.37	2.06	2.40	2.62	2.88	3.51	5731	1
lp__	-46.14	0.03	1.57	-50.22	-46.83	-45.78	-45.01	-44.25	3881	1

Samples were drawn using NUTS(diag_e) at Thu Mar 13 11:26:51 2025.

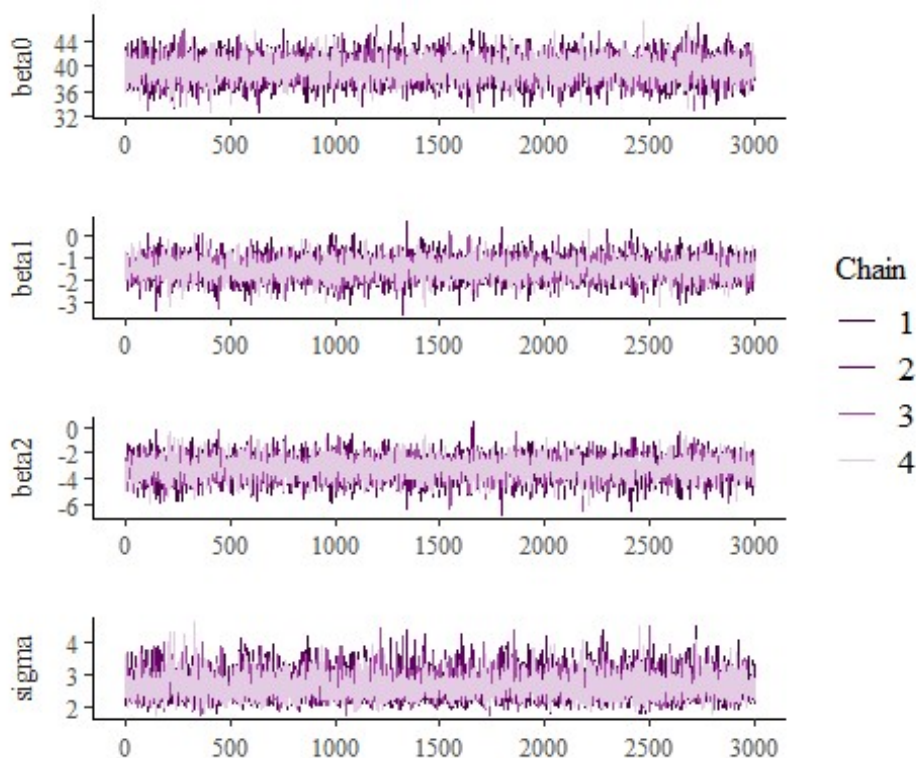
For each parameter, n_eff is a crude measure of effective sample size, and Rhat is the potential scale reduction factor on split chains (at convergence, Rhat=1).

```
post.hc <- as.array(stan_fit.hc)
```

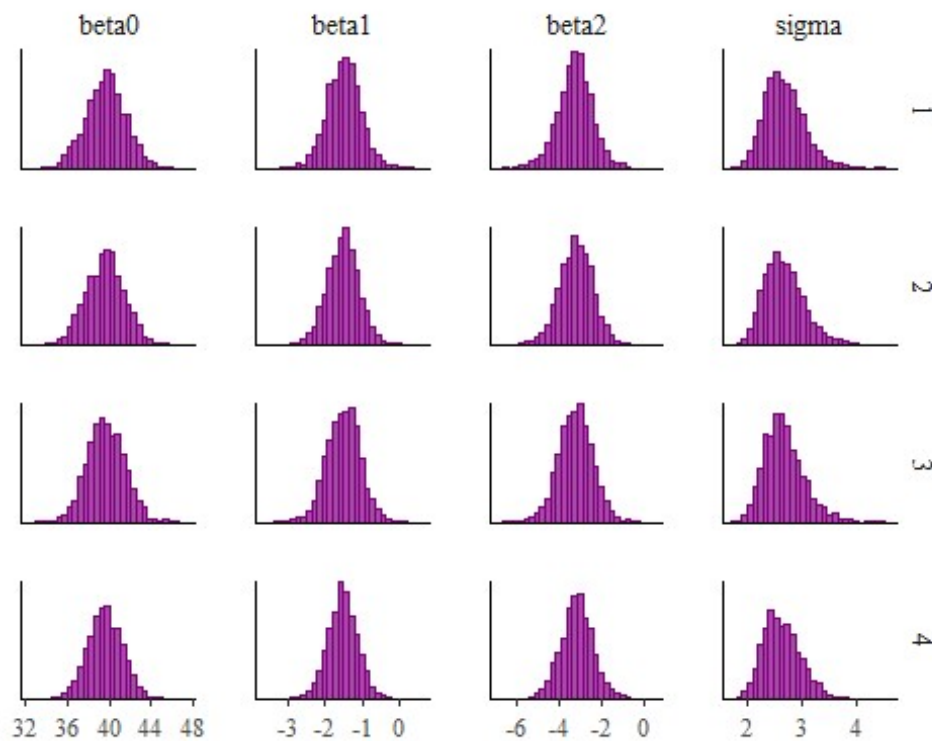
```
#Quality
```

```
color_scheme_set("purple")
```

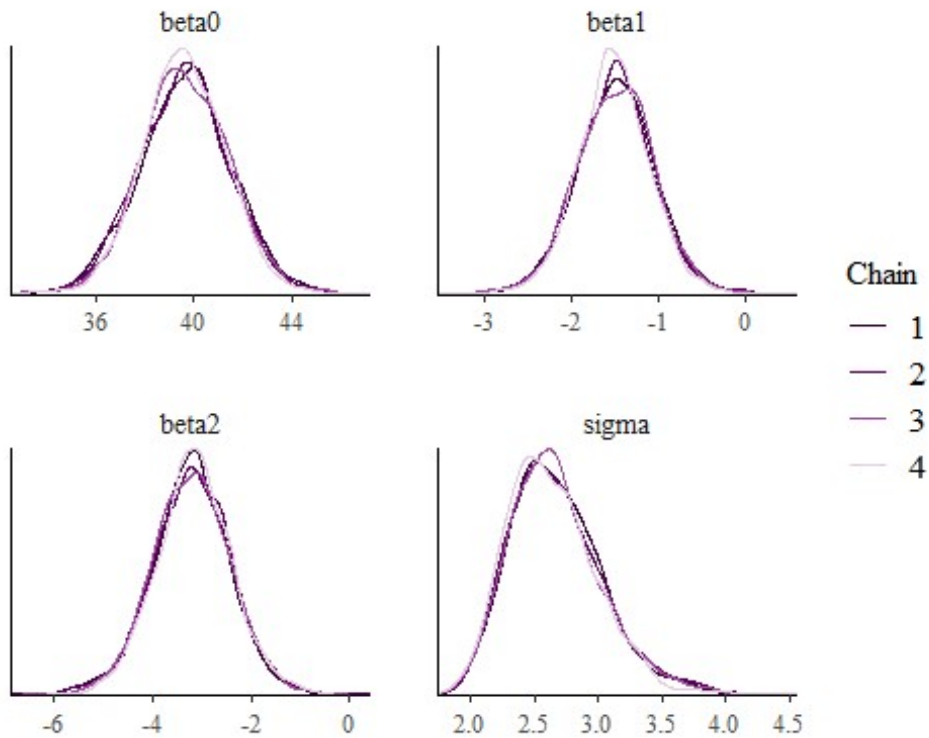
```
mcmc_trace(post.hc, pars = c("beta0", "beta1", "beta2", "sigma"),  
           facet_args = list(ncol = 1, strip.position = "left"))
```



```
mcmc_hist_by_chain(post.hc, pars = c("beta0", "beta1", "beta2", "sigma"))  
`stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```



```
mcmc_dens_overlay(post.hc, pars = c("beta0", "beta1", "beta2", "sigma"))
```



Analyze results - classic statistics

```
lm_results <- lm(formula = mpg ~ cyl + wt, data = mtcars)
summary(lm_results)$coefficients
```

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	39.686261	1.7149840	23.140893	3.043182e-20
cyl	-1.507795	0.4146883	-3.635972	1.064282e-03
wt	-3.190972	0.7569065	-4.215808	2.220200e-04

Analyze results - Bayesian statistics

Extract parameter names

```
param_names <- colnames(post.hc)
```

Compute mean and sd for each parameter

```
means <- colMeans(post.hc)
```

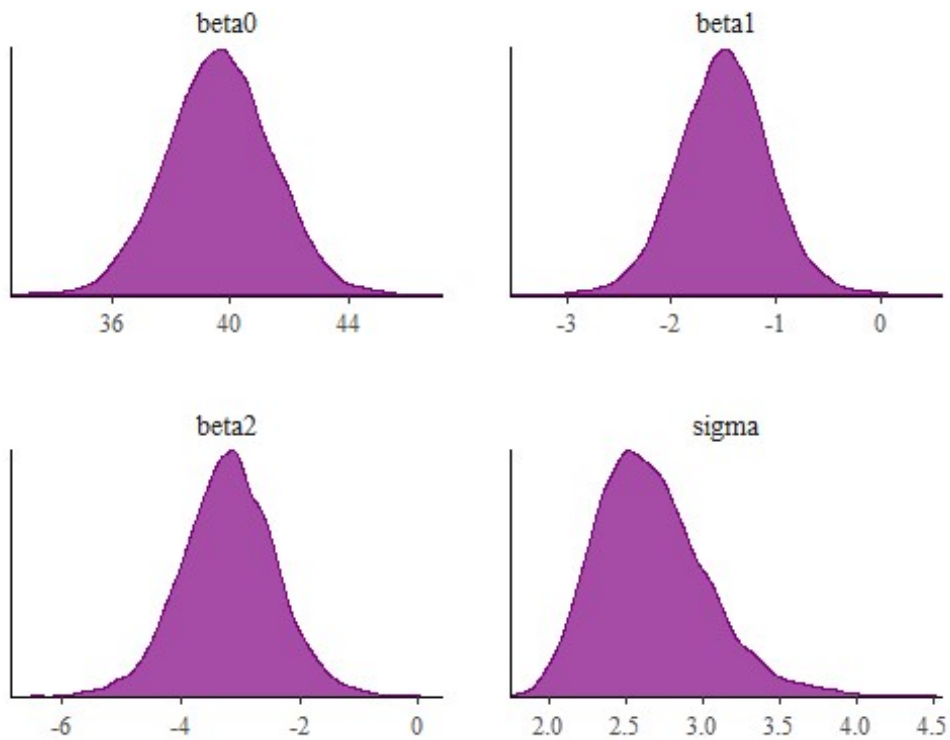
Display the results

```
param_statistics <- data.frame(Parameter = param_names, Mean = means[,1:5])
print(param_statistics)
```

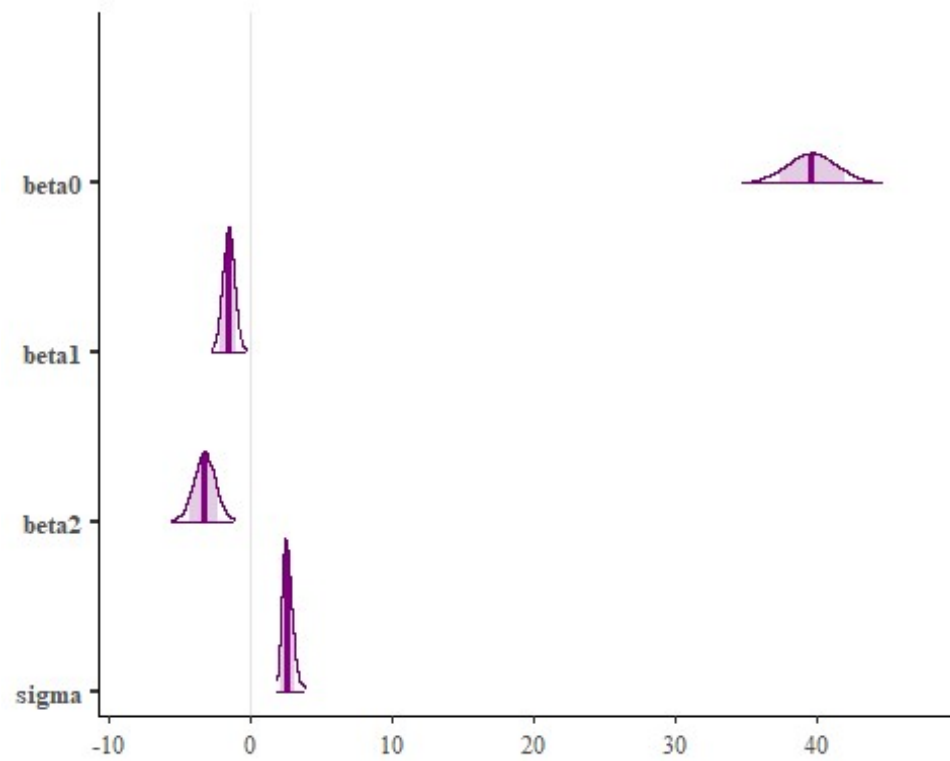
	Parameter	Mean.beta0	Mean.beta1	Mean.beta2	Mean.sigma	Mean.lp__
chain:1	chain:1	39.67896	-1.482971	-3.234667	2.675533	-46.16023
chain:2	chain:2	39.59355	-1.499829	-3.181124	2.680178	-46.22532
chain:3	chain:3	39.66647	-1.493362	-3.210846	2.671072	-46.18998
chain:4	chain:4	39.60903	-1.514229	-3.152530	2.638100	-45.98405

```
# Density plot
```

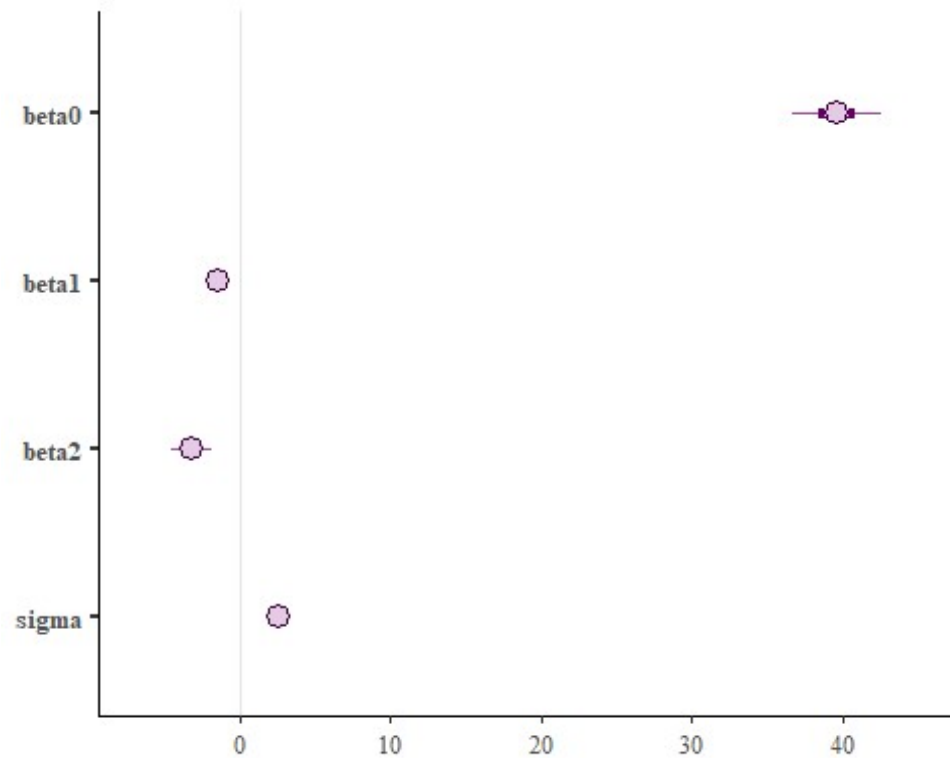
```
mcmc_dens(post.hc, pars = c("beta0", "beta1", "beta2", "sigma"))
```



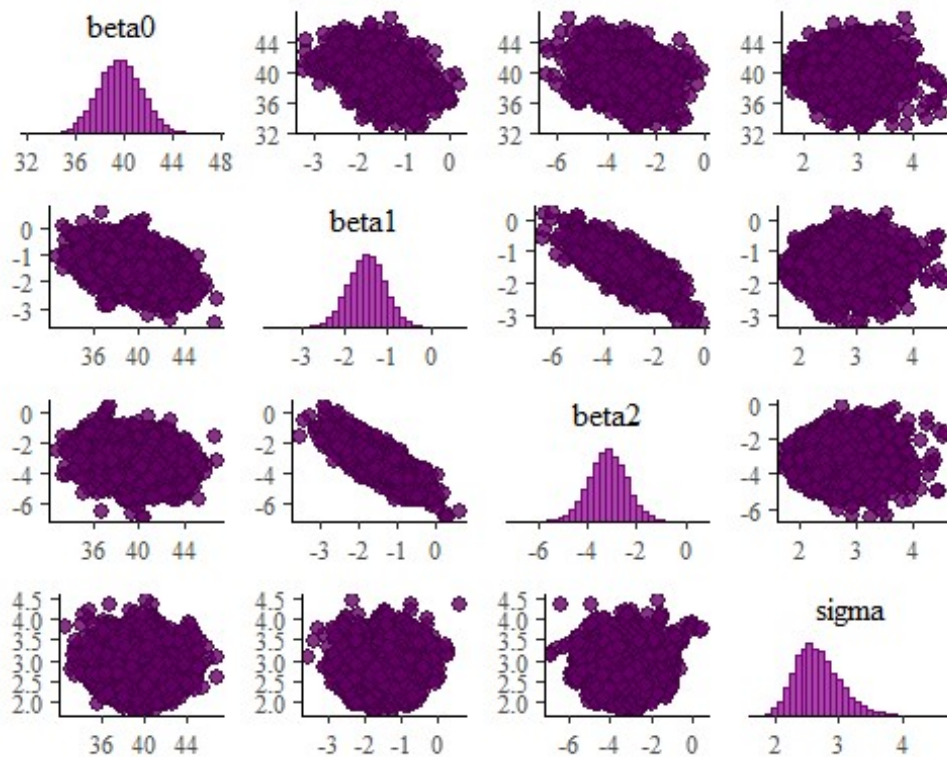
```
mcmc_areas(  
  post.hc,  
  pars = c("beta0", "beta1", "beta2", "sigma"),  
  prob = 0.8, # 80% intervals  
  prob_outer = 0.99, # 99%  
  point_est = "mean"  
)
```



```
mcmc_intervals(post.hc, pars=c("beta0", "beta1", "beta2", "sigma"))
```



```
# Pairs plot
mcmc_pairs(post.hc, pars = c("beta0", "beta1", "beta2", "sigma"))
```



3. The Student t multiple regression

Now we introduce a new regression model that assumes data with heavier tails, i.e. errors follow a Student-t distribution.

3.1 Student t distribution

```
# Student t distribution
# dt(x,df)
```

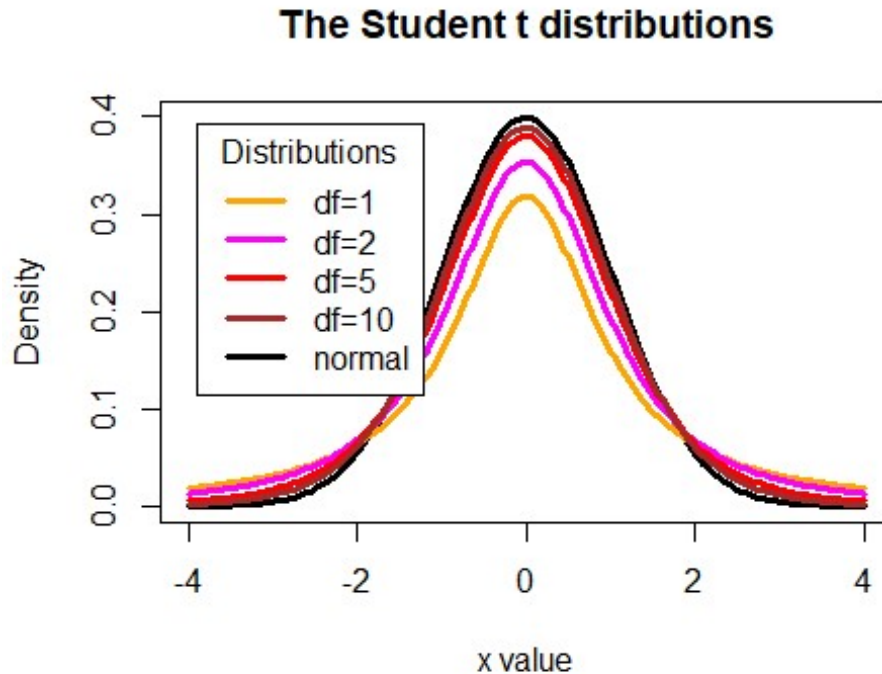
```
x <- seq(-4,4,.05)
hx <- dnorm(x)
```

```
degf <- c(1, 2, 5, 10)
colors <- c("orange", "magenta", "red", "brown", "black")
labels <- c("df=1", "df=2", "df=5", "df=10", "normal")
```

```
plot(x, hx, type="l", lty=1, lwd = 3, xlab="x value", col=colors[5],
     ylab="Density", main="The Student t distributions")
```

```
for (i in 1:4){
  lines(x, dt(x,degf[i]), lwd=3, col=colors[i])
}
```

```
legend("topleft", inset=.05, title="Distributions",
      labels, lwd=3, col=colors)
```



The t distribution is symmetric and the shape depends on the degrees of freedom. We can assume it is fixed (e.g., in classic statistics), but the Bayesian approach allows to assume we do not know df. Thus, df is a parameters and a prior needs to be specified. We can use the gamma distribution (df can be non-integer numbers).

3.2 Gamma distribution

```
# Gamma distribution
# dgamma(x, shape, rate)
```

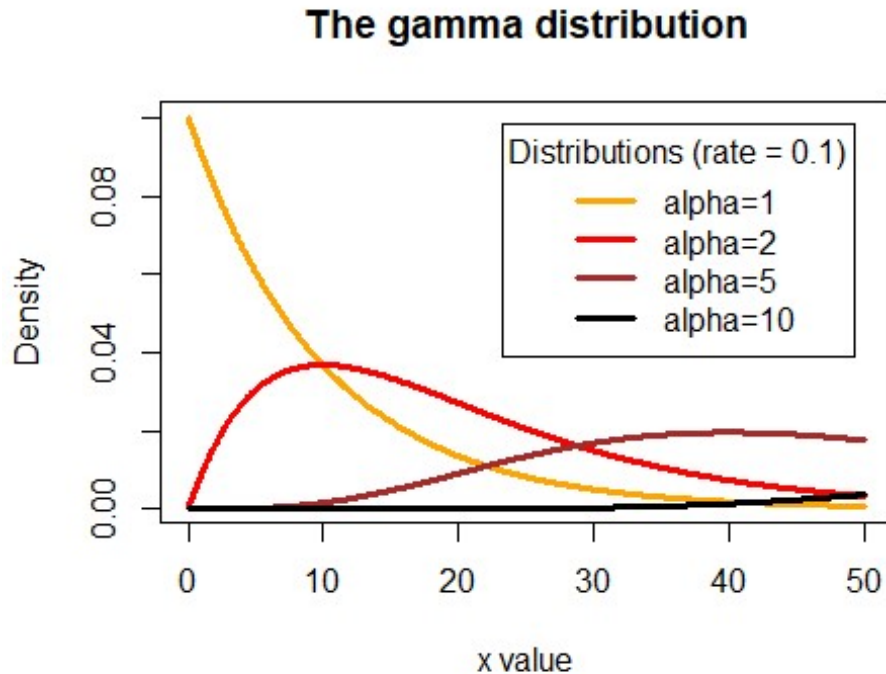
```
x <- seq(0,50,.1)
hx <- dgamma(x,1,0.1)

shape <- c(1, 2, 5, 10)
colors <- c("orange","red","brown","black")
labels <- c("alpha=1", "alpha=2", "alpha=5", "alpha=10")

plot(x, hx, type="l", lty=1, lwd = 3, xlab="x value",
     ylab="Density", main="The gamma distribution")

for (i in 1:4){
  lines(x, dgamma(x,shape[i],0.1), lwd=3, col=colors[i])
}
```

```
legend("topright", inset=.05, title="Distributions (rate = 0.1)",
      labels, lwd=3, col=colors)
```



3.3 Model specification

The likelihood model is defined by:

$$mpg_i | cyl_i, wt_i, \beta_0, \beta_1, \beta_2, \sigma, \nu \sim t(\mu_i, \sigma, \nu)$$

with $\mu_i = \beta_0 + \beta_1 cyl + \beta_2 wt$.

Now we have to set the priors for $\beta_0, \beta_1, \beta_2, \sigma, \nu$, that can be

$$(\beta_0, \beta_1, \beta_2) \sim N(0, 100^2 I_3)$$

$$\sigma \sim Half-Cauchy(0, 5)$$

$$\nu \sim Gamma(2, 0.1)$$

3.4 Model estimation in Stan

```
# Stan model code
```

```
stan_model.t <- "
```

```
// Linear Model with Student-t Errors
```

```

data {
  int N;                      // number of observations
  vector[N] y;                // response
  int K;                      // number of columns in the design
  matrix X                    // design matrix X (no intercept)
  matrix [N, K] X;
}

parameters {
  real alpha;                 // regression intercept
  vector[K] beta;             // regression coefficient vector
  real<lower = 0> sigma;       // dispersion
  real<lower = 2> nu;          // degrees of freedom (df >= 2 so
  that                         // there is a finite variance)
}

transformed parameters {
  vector[N] mu;
  mu = alpha + X * beta;
}

model {
  // Prior for betas
  alpha ~ normal(0.0, 100);
  beta ~ normal(0.0, 100);

  // Prior for sigma
  sigma ~ cauchy(0,5);

  // Prior for df
  nu ~ gamma(2, 0.1);         // Stan prior suggestion

  // likelihood
  y ~ student_t(nu, mu, sigma);
}"

# Running stan code

model.t <- stan_model(model_code = stan_model.t)

# Create a list of data for Stan

stan_data <- list(N=32, K=2, y=mtcars$mpg, X=cbind(mtcars$cyl,mtcars$wt))

# Fit the model to the data

stan_fit.t <- sampling(model.t, data = stan_data, chains = 4, iter = 10000)

```

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 1).

Chain 1:

Chain 1: Gradient evaluation took 6.2e-05 seconds

Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 0.62 seconds.

Chain 1: Adjust your expectations accordingly!

Chain 1:

Chain 1:

Chain 1: Iteration: 1 / 10000 [0%] (Warmup)

Chain 1: Iteration: 1000 / 10000 [10%] (Warmup)

Chain 1: Iteration: 2000 / 10000 [20%] (Warmup)

Chain 1: Iteration: 3000 / 10000 [30%] (Warmup)

Chain 1: Iteration: 4000 / 10000 [40%] (Warmup)

Chain 1: Iteration: 5000 / 10000 [50%] (Warmup)

Chain 1: Iteration: 5001 / 10000 [50%] (Sampling)

Chain 1: Iteration: 6000 / 10000 [60%] (Sampling)

Chain 1: Iteration: 7000 / 10000 [70%] (Sampling)

Chain 1: Iteration: 8000 / 10000 [80%] (Sampling)

Chain 1: Iteration: 9000 / 10000 [90%] (Sampling)

Chain 1: Iteration: 10000 / 10000 [100%] (Sampling)

Chain 1:

Chain 1: Elapsed Time: 1.433 seconds (Warm-up)

Chain 1: 1.603 seconds (Sampling)

Chain 1: 3.036 seconds (Total)

Chain 1:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 2).

Chain 2:

Chain 2: Gradient evaluation took 1.2e-05 seconds

Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 0.12 seconds.

Chain 2: Adjust your expectations accordingly!

Chain 2:

Chain 2:

Chain 2: Iteration: 1 / 10000 [0%] (Warmup)

Chain 2: Iteration: 1000 / 10000 [10%] (Warmup)

Chain 2: Iteration: 2000 / 10000 [20%] (Warmup)

Chain 2: Iteration: 3000 / 10000 [30%] (Warmup)

Chain 2: Iteration: 4000 / 10000 [40%] (Warmup)

Chain 2: Iteration: 5000 / 10000 [50%] (Warmup)

Chain 2: Iteration: 5001 / 10000 [50%] (Sampling)

Chain 2: Iteration: 6000 / 10000 [60%] (Sampling)

Chain 2: Iteration: 7000 / 10000 [70%] (Sampling)

Chain 2: Iteration: 8000 / 10000 [80%] (Sampling)

Chain 2: Iteration: 9000 / 10000 [90%] (Sampling)

Chain 2: Iteration: 10000 / 10000 [100%] (Sampling)

Chain 2:

Chain 2: Elapsed Time: 1.355 seconds (Warm-up)

Chain 2: 1.442 seconds (Sampling)

Chain 2: 2.797 seconds (Total)
Chain 2:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 3).

Chain 3:
Chain 3: Gradient evaluation took 1.2e-05 seconds
Chain 3: 1000 transitions using 10 leapfrog steps per transition would take 0.12 seconds.
Chain 3: Adjust your expectations accordingly!
Chain 3:
Chain 3:
Chain 3: Iteration: 1 / 10000 [0%] (Warmup)
Chain 3: Iteration: 1000 / 10000 [10%] (Warmup)
Chain 3: Iteration: 2000 / 10000 [20%] (Warmup)
Chain 3: Iteration: 3000 / 10000 [30%] (Warmup)
Chain 3: Iteration: 4000 / 10000 [40%] (Warmup)
Chain 3: Iteration: 5000 / 10000 [50%] (Warmup)
Chain 3: Iteration: 5001 / 10000 [50%] (Sampling)
Chain 3: Iteration: 6000 / 10000 [60%] (Sampling)
Chain 3: Iteration: 7000 / 10000 [70%] (Sampling)
Chain 3: Iteration: 8000 / 10000 [80%] (Sampling)
Chain 3: Iteration: 9000 / 10000 [90%] (Sampling)
Chain 3: Iteration: 10000 / 10000 [100%] (Sampling)
Chain 3:
Chain 3: Elapsed Time: 1.191 seconds (Warm-up)
Chain 3: 1.502 seconds (Sampling)
Chain 3: 2.693 seconds (Total)
Chain 3:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 4).

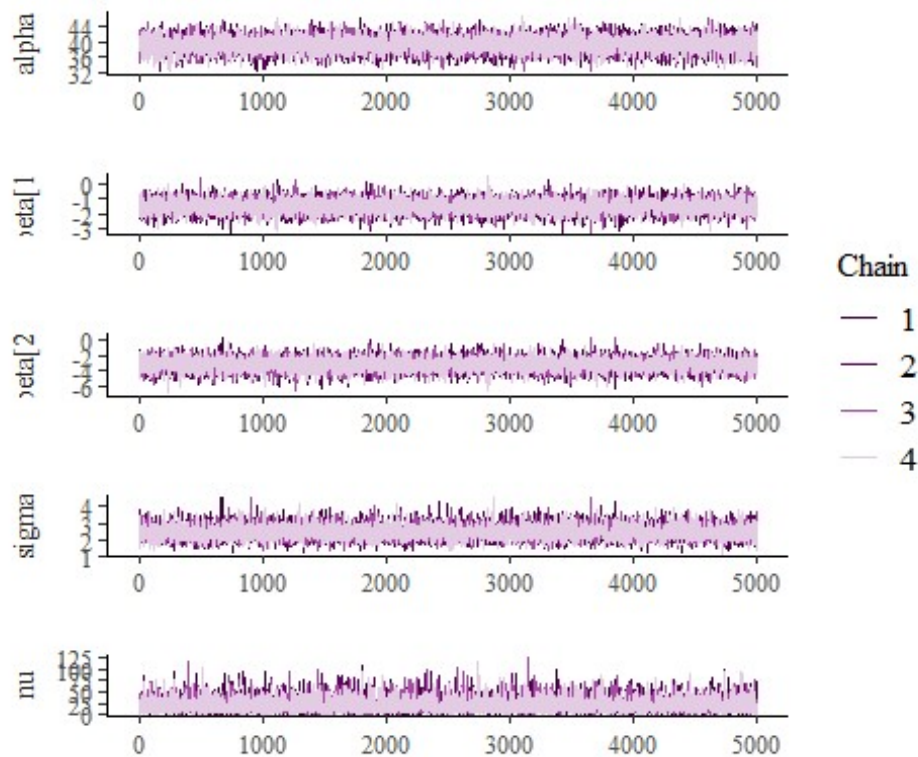
Chain 4:
Chain 4: Gradient evaluation took 1.2e-05 seconds
Chain 4: 1000 transitions using 10 leapfrog steps per transition would take 0.12 seconds.
Chain 4: Adjust your expectations accordingly!
Chain 4:
Chain 4:
Chain 4: Iteration: 1 / 10000 [0%] (Warmup)
Chain 4: Iteration: 1000 / 10000 [10%] (Warmup)
Chain 4: Iteration: 2000 / 10000 [20%] (Warmup)
Chain 4: Iteration: 3000 / 10000 [30%] (Warmup)
Chain 4: Iteration: 4000 / 10000 [40%] (Warmup)
Chain 4: Iteration: 5000 / 10000 [50%] (Warmup)
Chain 4: Iteration: 5001 / 10000 [50%] (Sampling)
Chain 4: Iteration: 6000 / 10000 [60%] (Sampling)
Chain 4: Iteration: 7000 / 10000 [70%] (Sampling)
Chain 4: Iteration: 8000 / 10000 [80%] (Sampling)
Chain 4: Iteration: 9000 / 10000 [90%] (Sampling)
Chain 4: Iteration: 10000 / 10000 [100%] (Sampling)
Chain 4:

```
Chain 4: Elapsed Time: 1.113 seconds (Warm-up)
Chain 4: 1.518 seconds (Sampling)
Chain 4: 2.631 seconds (Total)
Chain 4:
```

```
post <- as.array(stan_fit.t)
```

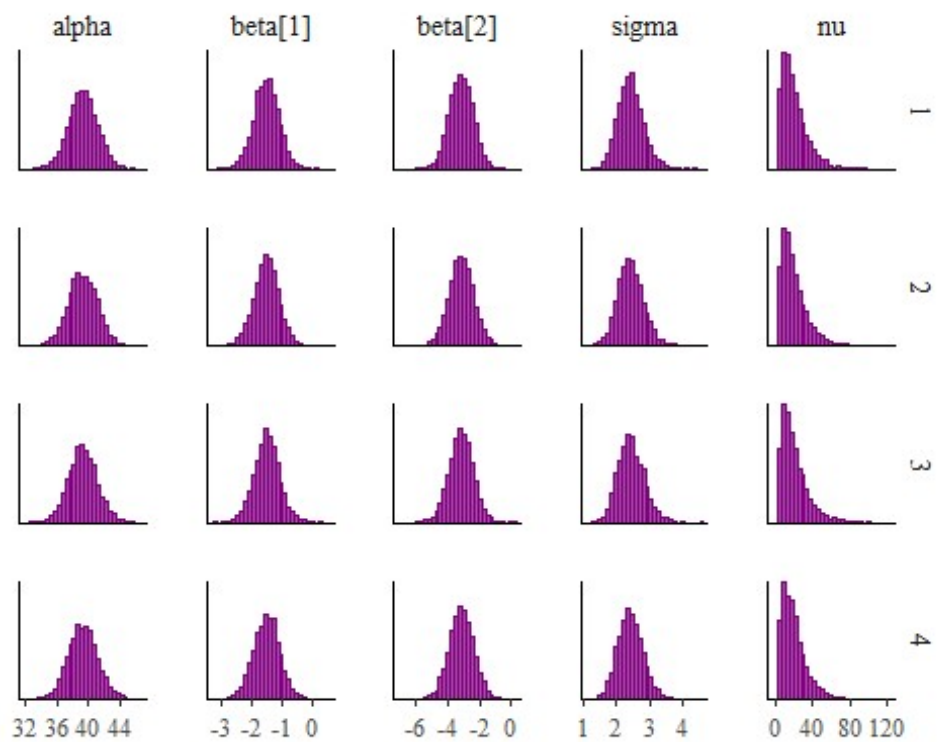
Kernel density plots of posterior draws with chains separated but overlaid on a single plot.

```
#Quality
color_scheme_set("purple")
mcmc_trace(post, pars = c("alpha", "beta[1]", "beta[2]", "sigma", "nu"),
            facet_args = list(ncol = 1, strip.position = "left"))
```

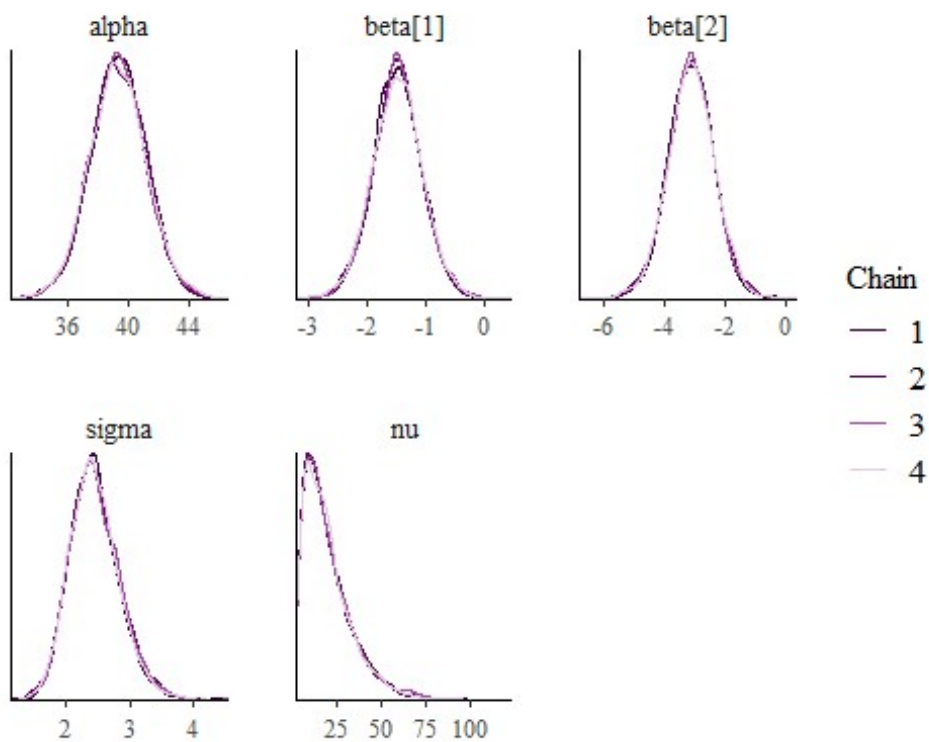


```
mcmc_hist_by_chain(post, pars = c("alpha", "beta[1]",
"beta[2]", "sigma", "nu"))
```

``stat_bin()`` using ``bins = 30``. Pick better value with ``binwidth``.



```
mcmc_dens_overlay(post, pars = c("alpha", "beta[1]", "beta[2]", "sigma", "nu"))
```



```
# Analyze results - classic statistics
```

```
summary(lm_results)$coefficients
```

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	39.686261	1.7149840	23.140893	3.043182e-20
cyl	-1.507795	0.4146883	-3.635972	1.064282e-03
wt	-3.190972	0.7569065	-4.215808	2.220200e-04

```
# Analyze results - classic statistics
```

```
print(stan_fit.t)
```

Inference for Stan model: anon_model.

4 chains, each with iter=10000; warmup=5000; thin=1;

post-warmup draws per chain=5000, total post-warmup draws=20000.

	mean	se_mean	sd	2.5%	25%	50%	75%	97.5%	n_eff	Rhat
alpha	39.33	0.02	1.85	35.66	38.10	39.32	40.57	43.01	12568	1
beta[1]	-1.50	0.00	0.42	-2.33	-1.77	-1.50	-1.23	-0.66	8679	1
beta[2]	-3.14	0.01	0.76	-4.66	-3.63	-3.14	-2.63	-1.65	9327	1
sigma	2.45	0.00	0.39	1.76	2.18	2.42	2.69	3.29	10590	1
nu	19.95	0.11	13.85	3.56	9.71	16.56	26.61	55.29	15152	1
mu[1]	22.12	0.01	0.62	20.89	21.71	22.12	22.53	23.35	13279	1
mu[2]	21.32	0.00	0.52	20.31	20.98	21.32	21.66	22.34	16070	1
mu[3]	26.06	0.01	0.79	24.50	25.54	26.06	26.57	27.63	12695	1
mu[4]	20.25	0.00	0.48	19.33	19.93	20.25	20.57	21.22	18165	1
mu[5]	16.55	0.01	0.77	15.04	16.04	16.54	17.06	18.11	11715	1
mu[6]	19.48	0.00	0.53	18.45	19.13	19.48	19.83	20.55	15467	1
mu[7]	16.14	0.01	0.73	14.72	15.66	16.14	16.61	17.60	12679	1
mu[8]	23.33	0.01	1.03	21.30	22.65	23.32	24.00	25.40	9847	1
mu[9]	23.45	0.01	1.01	21.46	22.79	23.44	24.11	25.48	9919	1
mu[10]	19.55	0.00	0.53	18.53	19.20	19.55	19.89	20.60	15716	1
mu[11]	19.55	0.00	0.53	18.53	19.20	19.55	19.89	20.60	15716	1
mu[12]	14.57	0.01	0.66	13.29	14.13	14.57	15.00	15.90	17390	1
mu[13]	15.64	0.01	0.68	14.31	15.19	15.64	16.08	17.01	14217	1
mu[14]	15.48	0.01	0.67	14.18	15.04	15.48	15.92	16.83	14758	1
mu[15]	10.87	0.01	1.17	8.56	10.11	10.87	11.63	13.18	12452	1
mu[16]	10.32	0.01	1.29	7.79	9.49	10.33	11.16	12.85	11983	1
mu[17]	10.57	0.01	1.23	8.15	9.77	10.57	11.37	13.00	12181	1
mu[18]	26.43	0.01	0.79	24.88	25.91	26.43	26.95	28.00	13050	1
mu[19]	28.27	0.01	0.95	26.39	27.65	28.27	28.90	30.14	12883	1
mu[20]	27.58	0.01	0.87	25.87	27.01	27.58	28.15	29.28	13221	1
mu[21]	25.60	0.01	0.80	24.04	25.07	25.60	26.12	27.21	12130	1
mu[22]	16.30	0.01	0.74	14.84	15.81	16.29	16.79	17.80	12277	1
mu[23]	16.57	0.01	0.78	15.05	16.05	16.56	17.07	18.13	11683	1
mu[24]	15.29	0.01	0.66	14.00	14.85	15.29	15.72	16.63	15468	1
mu[25]	15.28	0.01	0.66	13.98	14.83	15.27	15.71	16.61	15533	1
mu[26]	27.27	0.01	0.84	25.61	26.72	27.27	27.82	28.91	13291	1
mu[27]	26.62	0.01	0.80	25.05	26.09	26.62	27.15	28.20	13169	1
mu[28]	28.59	0.01	1.00	26.61	27.94	28.59	29.24	30.55	12684	1
mu[29]	17.40	0.01	0.90	15.62	16.80	17.39	17.98	19.19	10423	1

mu[30]	21.65	0.00	0.55	20.56	21.28	21.65	22.01	22.75	14794	1
mu[31]	16.14	0.01	0.73	14.72	15.66	16.14	16.61	17.60	12679	1
mu[32]	24.61	0.01	0.86	22.93	24.04	24.60	25.17	26.36	10888	1
lp__	-53.80	0.02	1.69	-57.99	-54.66	-53.45	-52.56	-51.55	6885	1

Samples were drawn using NUTS(diag_e) at Thu Mar 13 11:28:38 2025.
 For each parameter, n_eff is a crude measure of effective sample size,
 and Rhat is the potential scale reduction factor on split chains (at
 convergence, Rhat=1).

Analyze results - Bayesian statistics

Extract the posterior samples

```
post_samples <- as.array(stan_fit.t)
```

Extract parameter names

```
param_names <- colnames(post_samples)
```

Compute mean and sd for each parameter

```
means <- colMeans(post_samples)
```

```
sds <- apply(post_samples, 2, sd)
```

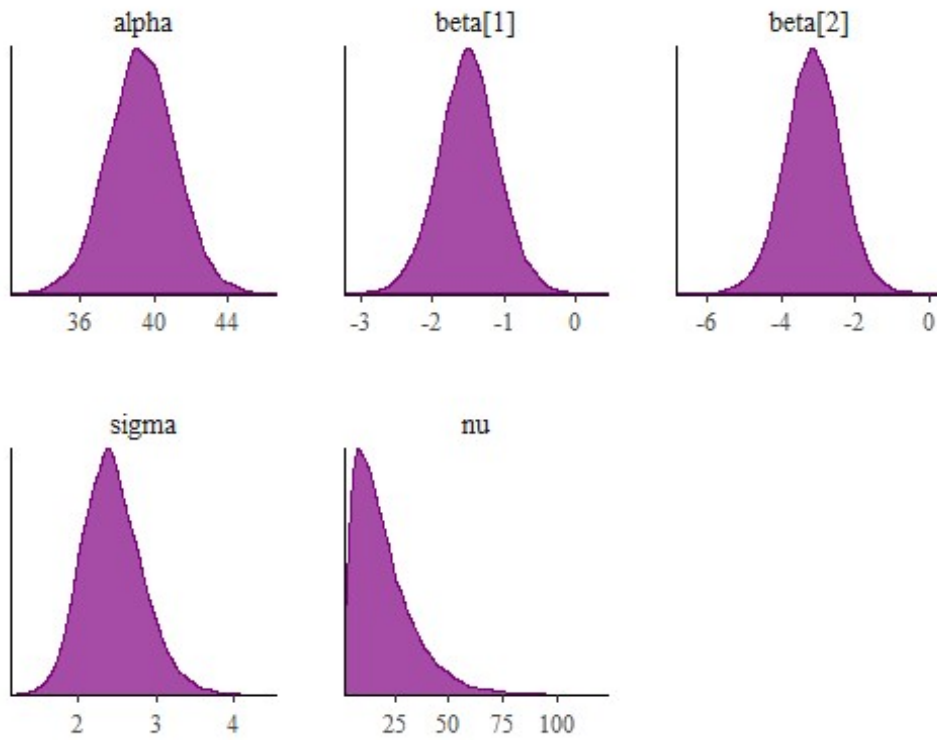
Display the results

```
param_statistics <- data.frame(Parameter = param_names, Mean = means[,1:5])
print(param_statistics)
```

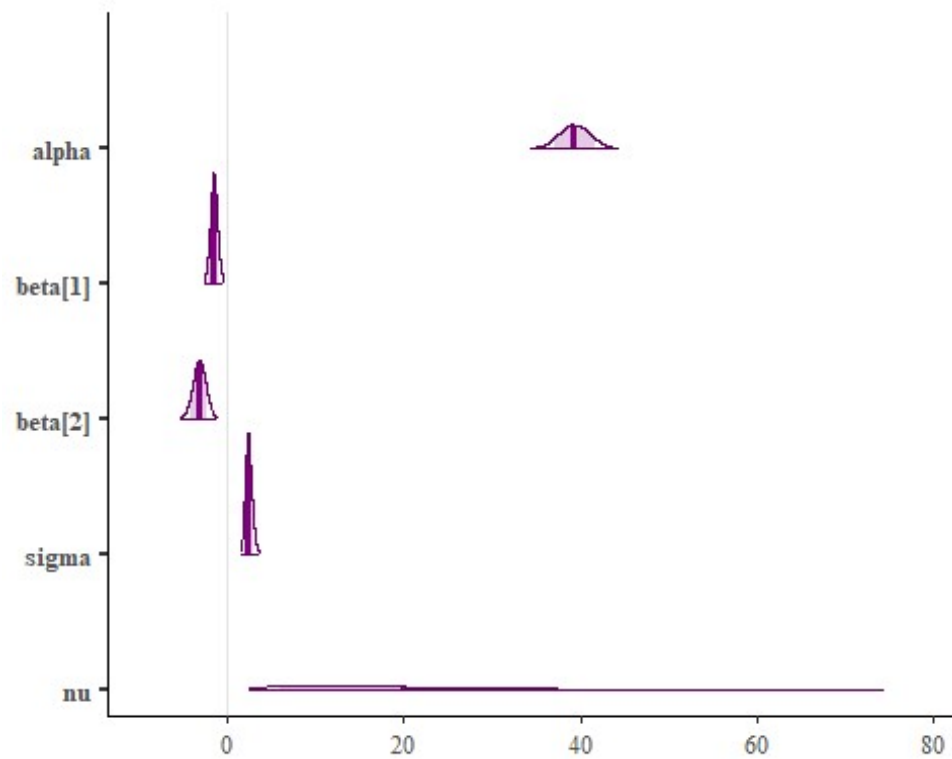
	Parameter	Mean.alpha	Mean.beta.1.	Mean.beta.2.	Mean.sigma	Mean.nu
chain:1	chain:1	39.36537	-1.500210	-3.144593	2.440667	20.07308
chain:2	chain:2	39.32897	-1.504253	-3.125442	2.443890	19.64905
chain:3	chain:3	39.30836	-1.487574	-3.149805	2.456322	20.22166
chain:4	chain:4	39.32788	-1.501011	-3.134320	2.441161	19.84928

Density plot

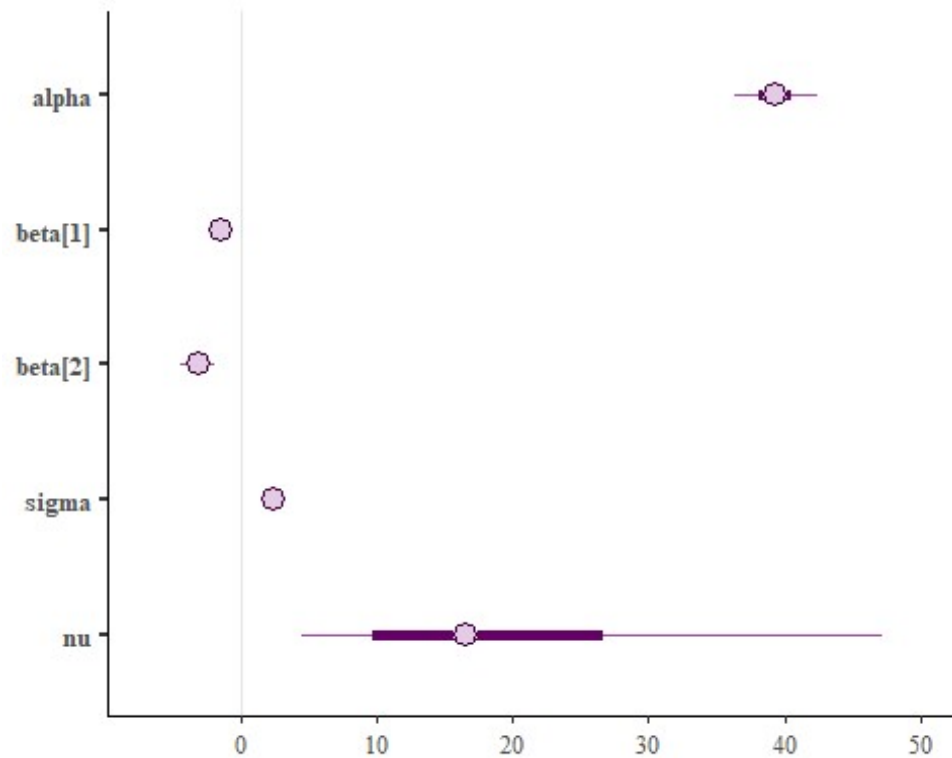
```
mcmc_dens(post, pars = c("alpha", "beta[1]", "beta[2]","sigma","nu"))
```



```
mcmc_areas(
  post,
  pars = c("alpha", "beta[1]", "beta[2]", "sigma", "nu"),
  prob = 0.8, # 80% intervals
  prob_outer = 0.99, # 99%
  point_est = "mean"
)
```



```
mcmc_intervals(post, pars=c("alpha", "beta[1]", "beta[2]", "sigma", "nu"))
```



```
# Pairs plot
```

```
mcmc_pairs(post, pars = c("alpha", "beta[1]", "beta[2]", "sigma", "nu"))
```

